

Payment Transaction Management System

Database Management System (DBMS) Mini Project

Name	:	Mohan Raj.S
Enrollment No.	:	TU-6253210211033
Week	:	5
Topic	:	Payment Transaction Management System

1. Introduction

The Payment Transaction Management System is a database application designed to record, track, and analyze payments made by customers through various modes such as UPI, Credit Card, Debit Card, Net Banking, and Cash. The system maintains the status of each transaction (Success, Failed, or Pending), allows management of successful and failed transactions separately, and provides analytical reports on payment methods and revenue trends. This project demonstrates core DBMS concepts including table design, constraints, DML operations, aggregate functions, joins, and report-oriented queries.

2. Objectives

- Design and create a Payment table to store transaction-level details.
- Capture essential attributes such as payment mode, transaction date, and status.
- Distinguish and manage successful and failed transactions efficiently.
- Analyze the payment methods most frequently used by customers.
- Generate summarized payment transaction reports for business insights.

3. System Description

The system is built around a central **Payment** table that stores one record per transaction. Each transaction is linked to a customer and an order, and records the mode of payment used, the amount involved, the date and time of the transaction, and the final status. Supporting tables for Customers and Orders provide referential context, allowing the system to answer questions such as "which customers use which payment modes" and "how many transactions failed today".

4. Entities and Relationships

Customer (1) --- (M) Order (1) --- (1) Payment : One customer can place many orders, and every order results in exactly one payment transaction record.

Entity	Key Attributes
Customer	customer_id (PK), name, email, phone
Orders	order_id (PK), customer_id (FK), order_date, order_amount
Payment	payment_id (PK), order_id (FK), customer_id (FK), payment_mode, payment_date, amount, status

5. Creating the Payment Table (DDL)

The following SQL statements create the supporting Customer and Orders tables, followed by the main Payment table with appropriate primary keys, foreign keys, and a CHECK constraint on the status column.

```
CREATE TABLE Customer (  
  customer_id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50) NOT NULL,  
  email VARCHAR(50) UNIQUE,  
  phone VARCHAR(15)  
);  
  
CREATE TABLE Orders (  
  order_id INT PRIMARY KEY AUTO_INCREMENT,  
  customer_id INT NOT NULL,  
  order_date DATE NOT NULL,  
  order_amount DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)  
);  
  
CREATE TABLE Payment (  
  payment_id INT PRIMARY KEY AUTO_INCREMENT,  
  order_id INT NOT NULL,  
  customer_id INT NOT NULL,  
  payment_mode VARCHAR(20) NOT NULL,  
  payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  amount DECIMAL(10,2) NOT NULL,  
  status VARCHAR(10) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),  
  CHECK (payment_mode IN ('UPI','Credit Card','Debit Card','Net Banking','Cash')),  
  CHECK (status IN ('Success','Failed','Pending'))  
);
```

6. Storing Payment Details (Sample Data)

Sample records showing payment mode, date, and status for a few transactions:

payment_id	order_id	customer_id	payment_mode	payment_date	amount	status
1	101	1	UPI	2026-08-20	1500.00	Success
2	102	2	Credit Card	2026-08-21	2500.00	Failed
3	103	1	Net Banking	2026-08-22	800.00	Success
4	104	3	Debit Card	2026-08-23	1200.00	Pending
5	105	4	UPI	2026-08-24	3000.00	Success
6	106	2	Cash	2026-08-24	500.00	Failed

```
INSERT INTO Payment (order_id, customer_id, payment_mode, payment_date, amount  
, status)  
VALUES  
(101, 1, 'UPI', '2026-08-20', 1500.00, 'Success'),  
(102, 2, 'Credit Card', '2026-08-21', 2500.00, 'Failed'),  
(103, 1, 'Net Banking', '2026-08-22', 800.00, 'Success'),  
(104, 3, 'Debit Card', '2026-08-23', 1200.00, 'Pending'),
```

```
(105, 4, 'UPI',      '2026-08-24', 3000.00, 'Success'),  
(106, 2, 'Cash',    '2026-08-24', 500.00,  'Failed');
```

7. Managing Successful and Failed Transactions

The queries below retrieve and separate transactions based on their status, which helps in reconciliation and follow-up on failed payments.

```
-- View all successful transactions
SELECT * FROM Payment WHERE status = 'Success';

-- View all failed transactions
SELECT * FROM Payment WHERE status = 'Failed';

-- Count of transactions by status
SELECT status, COUNT(*) AS total_transactions
FROM Payment
GROUP BY status;

-- Update a failed transaction after retry succeeds
UPDATE Payment
SET status = 'Success', payment_date = NOW()
WHERE payment_id = 2;

-- Delete very old failed/pending test records
DELETE FROM Payment
WHERE status = 'Failed' AND payment_date < '2025-01-01';
```

8. Analyzing Payment Methods Used by Customers

These queries help identify customer preferences and the popularity of each payment mode.

```
-- Most frequently used payment modes
SELECT payment_mode, COUNT(*) AS usage_count
FROM Payment
GROUP BY payment_mode
ORDER BY usage_count DESC;

-- Payment modes used by a specific customer
SELECT customer_id, payment_mode, COUNT(*) AS times_used
FROM Payment
WHERE customer_id = 1
GROUP BY payment_mode;

-- Success rate of each payment mode
SELECT payment_mode,
       SUM(CASE WHEN status = 'Success' THEN 1 ELSE 0 END) AS success_count,
       COUNT(*) AS total_count,
       ROUND(SUM(CASE WHEN status = 'Success' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2)
       AS success_rate_percent
FROM Payment
GROUP BY payment_mode;
```

9. Generating Payment Transaction Reports

The following queries generate summarized reports useful for business review, such as daily collections, customer-wise totals, and monthly revenue trends.

```
-- Daily transaction report
SELECT payment_date, COUNT(*) AS total_transactions,
```

```
        SUM(amount) AS total_amount
FROM Payment
GROUP BY payment_date
ORDER BY payment_date;

-- Customer-wise total successful payments
SELECT c.name, SUM(p.amount) AS total_paid
FROM Payment p
JOIN Customer c ON p.customer_id = c.customer_id
WHERE p.status = 'Success'
GROUP BY c.name
ORDER BY total_paid DESC;

-- Monthly revenue report
SELECT DATE_FORMAT(payment_date, '%Y-%m') AS month,
       SUM(amount) AS revenue
FROM Payment
WHERE status = 'Success'
GROUP BY DATE_FORMAT(payment_date, '%Y-%m')
ORDER BY month;
```

10. Conclusion

The Payment Transaction Management System successfully demonstrates how a relational database can be used to store payment records, manage transaction outcomes, and derive meaningful business insights through analytical queries. It applies fundamental DBMS concepts such as table creation with constraints, DML operations, joins, grouping, and conditional aggregation to build a practical, report-ready payment tracking solution.